```
    author, error = author_schema.load(data)
    result = author_schema.dump(author.create()).data
    return make_response(jsonify({"author": result}),200)

if __name__ == "__main__":
    app.run(debug=True)
```

So, we have now created and tested our sample Flask-MySQL CRUD application. We'll go over complex object relationships using Flask-SQLAlchemy in the later chapters, and next we'll create a similar Flask CRUD application using MongoEngine.

Sample Flask MongoEngine Application

MongoDB, as we discussed, is a powerful document-based NoSQL database. It uses a JSON-like document schema structure and is highly scalable. In this example, we'll create an Authors database CRUD application again, but this time we'll use MongoEngine rather than SQLAlchemy. MongoEngine adds MongoDB support for Flask and is quite similar to SQLAlchemy, but it lacks a couple of features due to the fact that MongoDB is still not widely used with Flask.

Let's get started with setting up our project for the flask-mongodb application. Just like the last time, create a new directory flask-mongodb and initiate a new virtual environment in there.

```
$ mkdir flask-mongodb && cd flask-mongodb
```

After creating the directory, let's spawn our virtual environment and activate it.

```
$ virtualenv venv
$ source venv/bin/activate
```

Now let's install our project dependencies using PIP.

```
(venv) $ pip install flask
```

We'll need Flask-MongoEngine and Flask-marshmallow, so let's install them as well.

```
(venv) $ pip install flask-mongoengine
(venv) $ pip install flask-marshmallow
```

After we are done installing the dependencies, we can create our app.py file and start writing the code.

So, the following code is the skeleton of the app where are import flask, create an app instance, and then import MongoEngine to create a db instance.

```
from flask import Flask, request, jsonify, make_response
from flask_mongoengine import MongoEngine
from marshmallow import Schema, fields, post_load
from bson import ObjectId

app = Flask(__name__)
app.config['MONGODB_DB'] = 'authors'
db = MongoEngine(app)

Schema.TYPE_MAPPING[ObjectId] = fields.String

if __name__ == "__main__":
    app.run(debug=True)
```

```
Here TYPE_MAAPPING helps marshmallow understand the ObjectId
type while serializing and de-serializing the data.
```

Note We don't need db.create_all() here since MongoDB will create it on the fly, during the first time you save the value in your collection.

If you now run the application, your server should start, but it'll have nothing to process but just create the db instance and make the connection. Next, let's create an author model using MongoEngine.

The code for creating the author model is fairly simple in this case and looks like this:

```
class Authors(db.Document):
    name = db.StringField()
    specialisation = db.StringField()
```

Let's now create the marshmallow schema which we'll need to dump our db objects into serialized JSON.

```
class AuthorsSchema(Schema):
    name = fields.String(required=True)
    specialisation = fields.String(required=True)
```

The preceding code lets us create the schema which we'll use to map our db object to marshmallow. Notice that here we are not using marshmallow-sqlalchemy which has an extra layer of support for SQLAlchemy and the code looks slightly changed due to that here.

Now we can write our GET endpoint to fetch all the authors from our DB.

```
@app.route('/authors', methods = ['GET'])
def index():
    get_authors = Authors.objects.all()
    author_schema =AuthorsSchema(many=True,only=['id','name',
    'specialisation'])
    authors, error = author_schema.dump(get_authors)
    return make_response(jsonify({"authors": authors}))
```

Note MongoEngine returns the unique ObjectId in "id" field which is autogenerated and hence not specified in the schema.

Now, let's start the application again using the following command.

```
(venv) $ python app.py
```

If there were no errors, you should see the following output, and your application should be up and running.

```
(venv) $ python app.py
* Serving Flask app "app" (lazy loading)
 * Environment: production
   WARNING: Do not use the development server in a production
   environment.
   Use a production WSGI server instead.
 * Debug mode: on
 * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 779-301-240
```

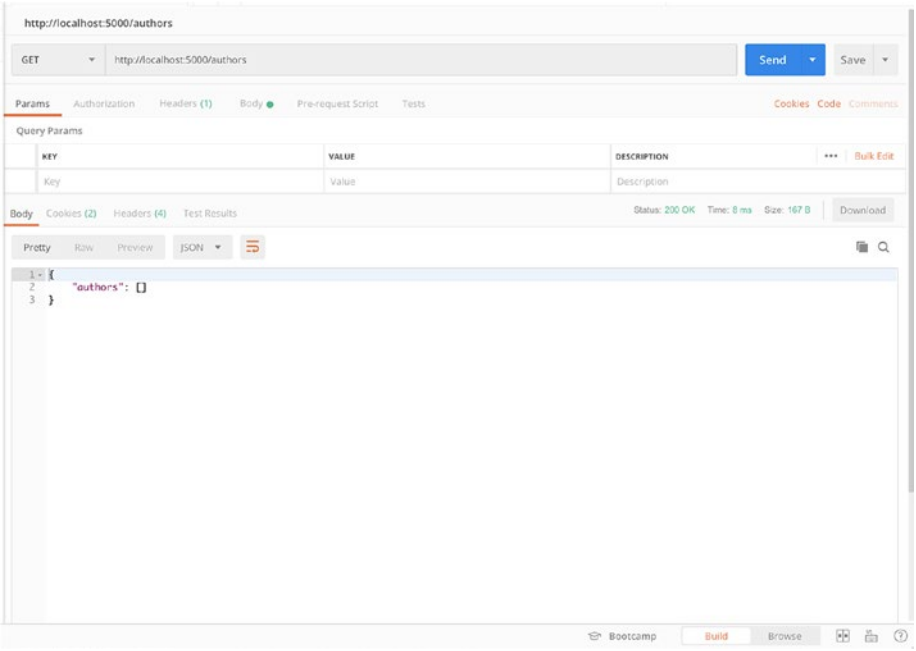

Figure 2-7. *Requesting GET /authors*

Now that our GET endpoint is working (Figure 2-7), let's create a POST /authors endpoint to register authors in the database.

```
@app.route('/authors', methods = ['POST'])
def create_author():
    data = request.get_json()
    author = Authors(name=data['name'],specialisation=data
    ['specialisation'])
    author.save()
    author_schema = AuthorsSchema(only=['name',
    'specialisation'])
    authors, error = author_schema.dump(author)
    return make_response(jsonify({"author": authors}),201)
```

The preceding code puts the request JSON data in data variable, creates an object of class Authors, and invokes the save() method on it. Next it creates a schema using the AuthorsSchema and dumps the new object to return it back to the user confirming the user was created with a 201 status code.

Now re-run the application and request the POST endpoint with sample author details to register.

We'll use the same JSON data to post to this application like we did in the other application.

```
{
      "name" : "Kunal Relan",
      "specialisation" : "Python"
}
```

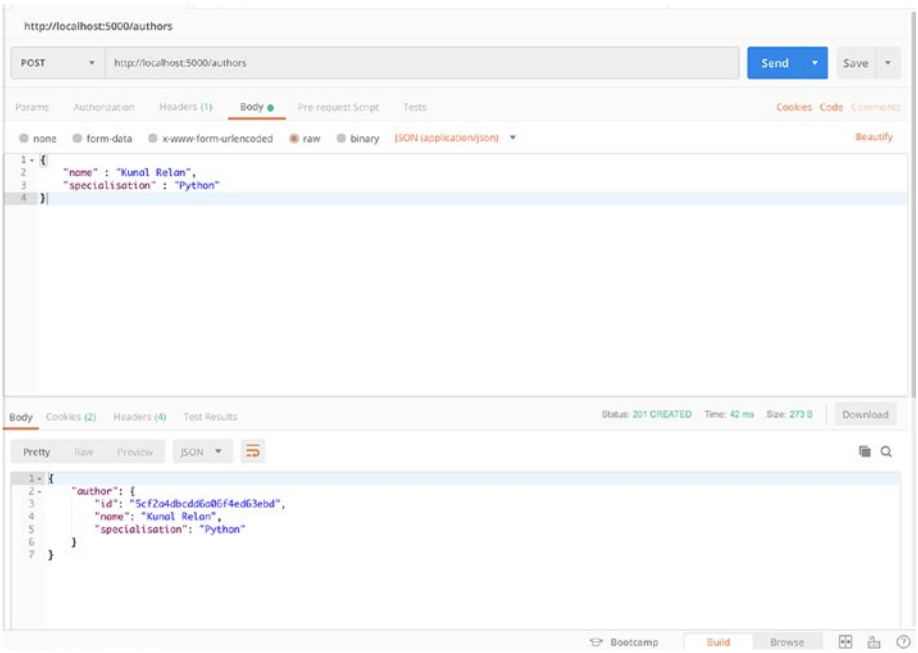

Figure 2-8. *Requesting POST /authors*

Upon requesting you should get a similar output to what you see in Figure 2-8, and now just to confirm that our GET endpoint works fine, we'll request it again to see if it returns the data.

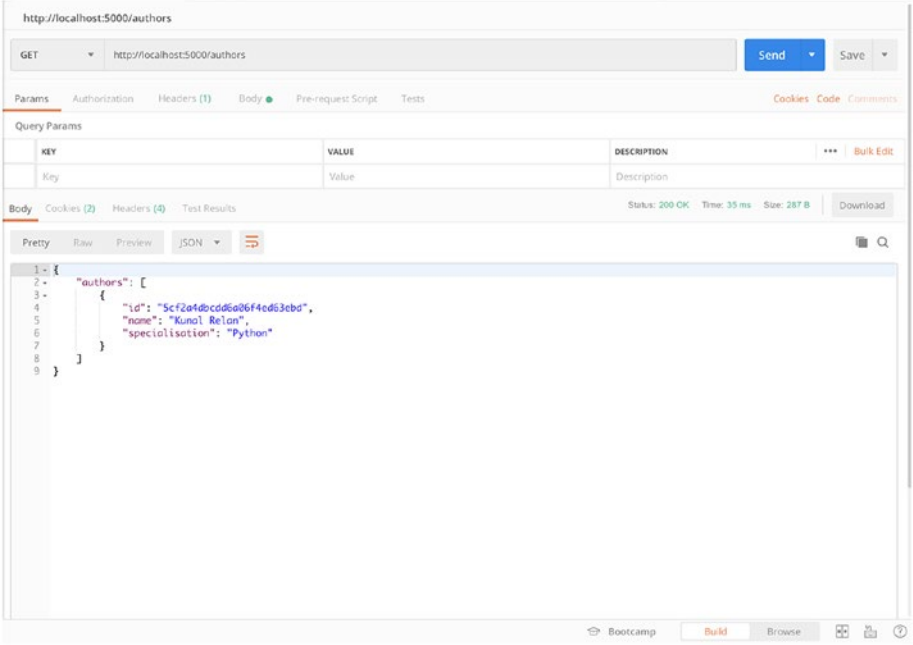

Figure 2-9. *Requesting GET /authors*

As you see in Figure 2-9, we get our recently registered author in the GET /authors endpoint.

Next we'll create an endpoint to return authors using the author ID and then update endpoint to update author details using author ID and the last endpoint to delete an author using author ID.

For GET author by ID, we'll have a route like /authors/<id> which will take author ID from the request parameter and find the matching author.

Add the following code for the GET author by ID endpoint below your GET all authors route.

```
@app.route('/authors/<id>', methods = ['GET'])
def get_author_by_id(id):
    get_author = Authors.objects.get_or_404(id=ObjectId(id))
    author_schema = AuthorsSchema(only=['id', 'name',
    'specialisation'])
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))
```

And now when you request the endpoint /authors/<id>, it shall return the user with the matching ObjectId (Figure 2-10).

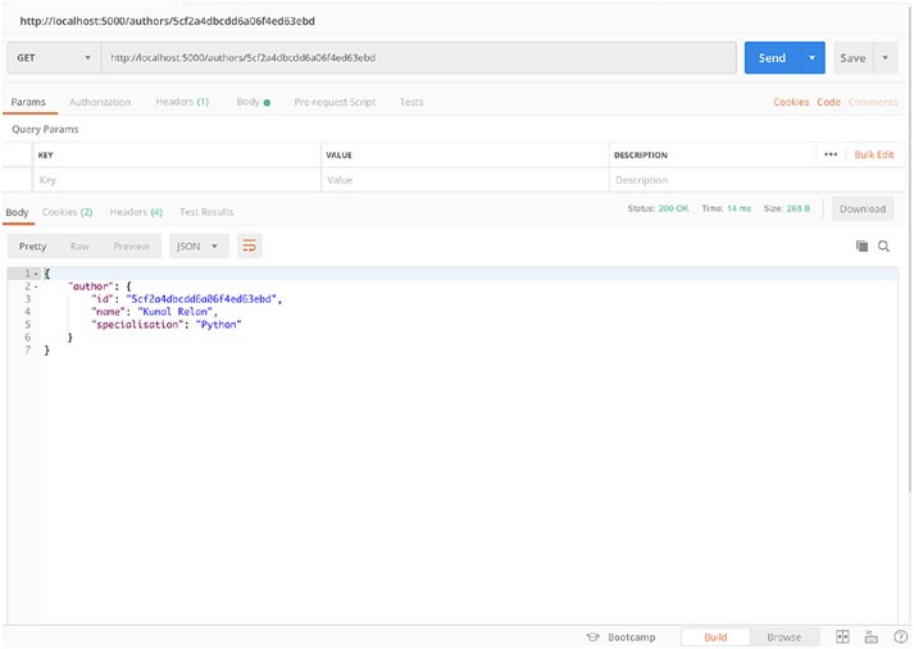

Figure 2-10. *GET author by ID*

So next we'll create PUT endpoint to update author info using author ID. Add the following code for the PUT author endpoint.

```
@app.route('/authors/<id>', methods = ['PUT'])
def update_author_by_id(id):
    data = request.get_json()
    get_author = Authors.objects.get(id=ObjectId(id))
    if data.get('specialisation'):
        get_author.specialisation = data['specialisation']
    if data.get('name'):
        get_author.name = data['name']
    get_author.save()
    get_author.reload()
    author_schema = AuthorsSchema(only=['id', 'name',
    'specialisation'])
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))
```

Open Postman and hit the same route as we did in the other module to update author info, but here use the ObjectID returned in the GET endpoint instead.

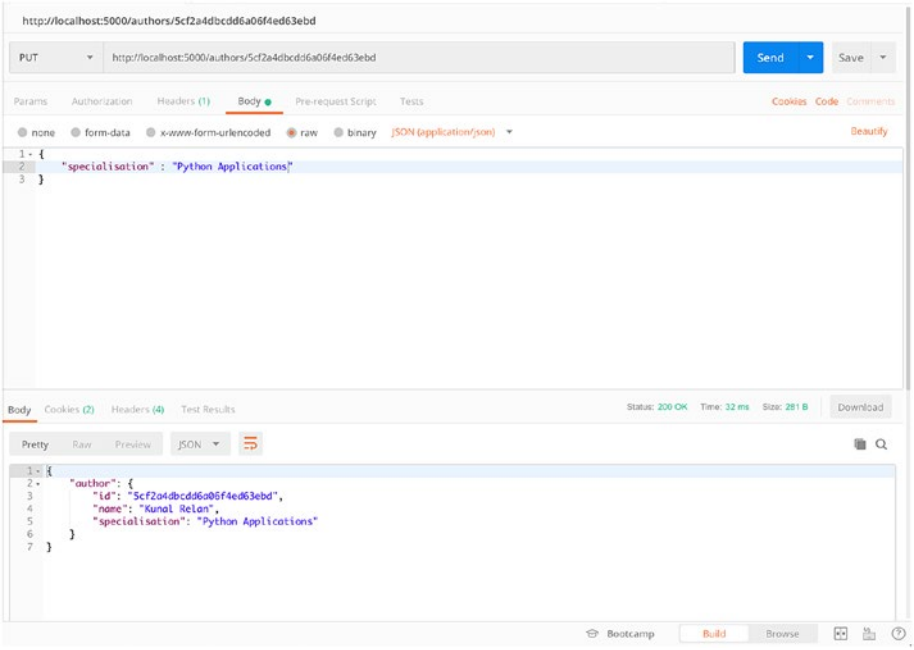

Figure 2-11. *PUT author endpoint*

As you can see in Figure 2-11, we were able to update the author specialisation using the PUT endpoint. Next we'll create the DELETE endpoint to delete an author using the author ID to complete our CRUD application.

Add the following code to create the DELETE endpoint to our application.

```
@app.route('/authors/<id>', methods = ['DELETE'])
def delete_author_by_id(id):
    Authors.objects(id=ObjectId(id)).delete()
    return make_response("",204)
```

Now let's delete our newly created author using the author ID, and similar to the last application, this endpoint will not return any data but a 204 status code.

Request the delete endpoint using the author ID you did previously, and it shall return a similar response as in Figure 2-12.

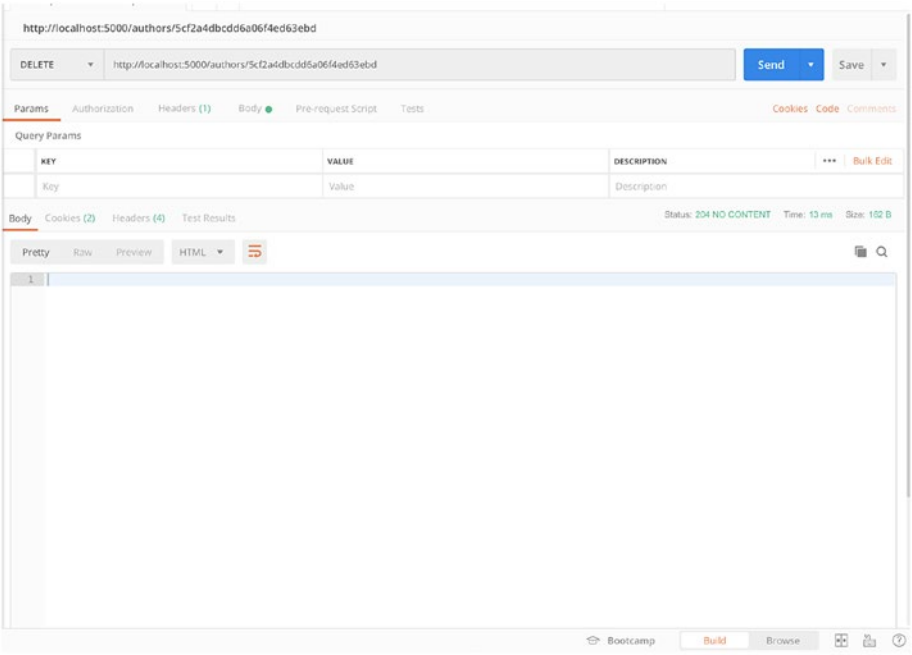

Figure 2-12. *DELETE author endpoint*

So this wraps up our flask-mongo CRUD application, and the final code in your app.py should look like this.

```
from flask import Flask, request, jsonify, make_response
from flask_mongoengine import MongoEngine
from marshmallow import Schema, fields, post_load
from bson import ObjectId

app = Flask(__name__)
app.config['MONGODB_DB'] = 'DB_NAME'
db = MongoEngine(app)

Schema.TYPE_MAPPING[ObjectId] = fields.String

class Authors(db.Document):
    name = db.StringField()
```

```
    specialisation = db.StringField()

class AuthorsSchema(Schema):
    name = fields.String(required=True)
    specialisation = fields.String(required=True)

@app.route('/authors', methods = ['GET'])
def index():
    get_authors = Authors.objects.all()
    author_schema = AuthorsSchema(many=True, only=['id',
    'name', 'specialisation'])
    authors, error = author_schema.dump(get_authors)
    return make_response(jsonify({"authors": authors}))

@app.route('/authors/<id>', methods = ['GET'])
def get_author_by_id(id):
    get_author = Authors.objects.get_or_404(id=ObjectId(id))
    author_schema = AuthorsSchema(only=['id', 'name',
    'specialisation'])
    author, error = author_schema.dump(get_author)
    return make_response(jsonify({"author": author}))

@app.route('/authors/<id>', methods = ['PUT'])
def update_author_by_id(id):
    data = request.get_json()
    get_author = Authors.objects.get(id=ObjectId(id))
    if data.get('specialisation'):
        get_author.specialisation = data['specialisation']
    if data.get('name'):
        get_author.name = data['name']
    get_author.save()
    get_author.reload()
```